

MayaScript

Mynor David Ruano Cabrera

3 de mayo de 2025

Índice

1. Introducción	4
1.1. Nombre del Lenguaje	4
1.2. Contexto del Lenguaje	4
2. Herramientas y Tecnologías Utilizadas	5
2.1. Lenguaje de Implementación	5
2.2. Herramientas Adicionales	5
3. Sintaxis del Lenguaje	5
3.1. Estructura Básica	6
3.2. Tipos de Datos	6
3.3. Variables y Constantes	6
3.3.1. Declaración de Variables	6
3.3.2. Declaración de Constantes	7
3.4. Operadores	7
3.4.1. Operadores Aritméticos	7
3.4.2. Operadores de Comparación	7
3.4.3. Operadores Lógicos	8
4. Estructuras de Control	8
4.1. Condicionales	8
4.2. Bucle while	8
4.3. Bucle for	9

5. Funciones	9
5.1. Declaración de Función	9
5.2. Retorno de Valores	9
5.3. Llamada a Función	9
6. Entrada y Salida	10
6.1. Salida de Datos	10
6.2. Entrada de Datos	10
7. Análisis Léxico	10
7.1. Expresiones Regulares	10
7.2. Tabla de Símbolos	10
7.3. Ejemplo de Tabla de Símbolos	11
7.4. Tabla de Errores	12
7.5. Tipos de Errores Léxicos	12
8. Análisis Sintáctico	13
8.1. Gramática BNF	13
8.1.1. Programa	13
8.1.2. Declaraciones	13
8.1.3. Tipos	13
8.1.4. Sentencias	14
8.1.5. Expresiones	14
8.1.6. Literales y Tokens básicos	15
8.2. Árboles de Derivación	15
8.2.1. Ejemplo 1: Declaración de variable con asignación . . .	15
8.2.2. Ejemplo 2: Sentencia if-else	15
8.2.3. Ejemplo 3: Bucle while	16
8.3. Tipos de Errores Sintácticos	17
8.4. Ejemplo de Tabla de Errores	17
9. Palabras Reservadas	19
10. Ejemplos Completos	19
10.1. Ejemplo 1: Calculadora Simple	19
10.2. Ejemplo 2: Factorial	20

11.Pruebas	21
11.1. Pruebas con Código Correcto	21
11.2. Pruebas con Código Erróneo	21
12.Modos Interactivos	22
12.1. Ejemplo de uso interactivo	22
13.Limitaciones Actuales	22
14.Conclusiones	22

1 Introducción

MayaScript es un lenguaje de programación inspirado en la cultura maya, diseñado como proyecto educativo para demostrar los principios fundamentales de los compiladores. Este documento presenta todos los aspectos del lenguaje, desde su concepción hasta su implementación técnica.

1.1 Nombre del Lenguaje

El nombre "MayaScript" combina la referencia a la civilización maya, una de las culturas más avanzadas de Mesoamérica conocida por sus logros en matemáticas, astronomía y escritura, con el sufijo "Script" que denota un lenguaje de programación. Esta combinación representa la filosofía del proyecto: crear un lenguaje que honre la rica tradición cultural maya mientras ofrece una plataforma educativa para comprender los conceptos modernos de compilación.

1.2 Contexto del Lenguaje

MayaScript fue desarrollado como un proyecto educativo con los siguientes objetivos:

- Demostrar los principios fundamentales de compilación e interpretación.
- Crear un lenguaje con sintaxis única inspirada en términos de la cultura maya.
- Proporcionar una plataforma para aprender las fases de un compilador: análisis léxico, sintáctico y semántico.
- Ofrecer una herramienta didáctica para comprender la estructura y el funcionamiento de los lenguajes de programación.

El contexto educativo del lenguaje guía todas las decisiones de diseño, desde la elección de palabras clave hasta la implementación de características específicas.

2 Herramientas y Tecnologías Utilizadas

2.1 Lenguaje de Implementación

MayaScript ha sido desarrollado utilizando Java, un lenguaje de programación orientado a objetos ampliamente utilizado. La elección de Java como lenguaje de implementación se basa en:

- Portabilidad: La máquina virtual de Java (JVM) permite que el compilador se ejecute en diferentes plataformas.
- Orientación a objetos: Facilita la estructuración del compilador en componentes claros y bien definidos.
- Bibliotecas estándar robustas: Java proporciona herramientas para manejo de cadenas, colecciones y E/S.
- Popularidad en entornos académicos: Facilita la comprensión del código por estudiantes familiarizados con Java.

2.2 Herramientas Adicionales

Además del lenguaje base, se utilizaron las siguientes herramientas:

- **Maven:** Sistema de gestión de dependencias y construcción del proyecto.
- **Git:** Sistema de control de versiones para gestionar el desarrollo colaborativo.

3 Sintaxis del Lenguaje

MayaScript presenta una sintaxis única inspirada en términos de la cultura maya, combinada con estructuras familiares de los lenguajes de programación modernos.

3.1 Estructura Básica

Todo programa en MayaScript debe comenzar con la palabra clave **mayab** y terminar con **ahau**:

```
1 mayab
2     // C digo del programa
3 ahau
```

Listing 1: Estructura básica de un programa MayaScript

Esta estructura delimita claramente el inicio y fin del programa, similares a las palabras clave **begin** y **end** en algunos lenguajes, pero con una identidad cultural distintiva.

3.2 Tipos de Datos

MayaScript soporta los siguientes tipos de datos básicos:

Tipo	Palabra clave	Descripción	Ejemplo
Entero	hunab	Números enteros	42
Flotante	kin	Números decimales	3.14
Cadena	nawal	Texto entre comillas dobles	"Hola"
Booleano	bool	Valores verdadero/falso	haab (true), manik (false)

Cuadro 1: Tipos de datos en MayaScript

3.3 Variables y Constantes

3.3.1. Declaración de Variables

Para declarar una variable se utiliza la palabra clave **tzolk** seguida del tipo de dato y el identificador:

```
1 tzolk hunab edad = 25;
2 tzolk nawal nombre = "Juan";
3 tzolk kin altura = 1.75;
4 tzolk bool activo = haab;
```

Listing 2: Declaración de variables

3.3.2. Declaración de Constantes

Para declarar una constante se utiliza la palabra clave **tun**, lo que indica que el valor no puede modificarse después de su inicialización:

```
1 tun hunab MAX_USUARIOS = 100;  
2 tun kin PI = 3.1416;
```

Listing 3: Declaración de constantes

3.4 Operadores

MayaScript admite diversos operadores para realizar operaciones en los datos:

3.4.1. Operadores Aritméticos

Operador	Descripción	Ejemplo
+	Suma	a + b
-	Resta	a - b
*	Multiplicación	a * b
/	División	a / b
%	Módulo	a % b

Cuadro 2: Operadores aritméticos

3.4.2. Operadores de Comparación

Operador	Descripción	Ejemplo
==	Igual a	a == b
!=	No igual a	a != b
>	Mayor que	a > b
<	Menor que	a < b
>=	Mayor o igual que	a >= b
<=	Menor o igual que	a <= b

Cuadro 3: Operadores de comparación

3.4.3. Operadores Lógicos

Operador	Descripción	Ejemplo
&&	AND lógico	a && b
——	OR lógico	a —— b
!	Negación	!a

Cuadro 4: Operadores lógicos

4 Estructuras de Control

MayaScript incorpora estructuras de control comunes en los lenguajes de programación, pero con nombres inspirados en la cultura maya.

4.1 Condicionales

La sentencia if-else se implementa con `chac-imix`:

```
1 chac (edad >= 18) {  
2     pakal "Eres mayor de edad";  
3 } imix {  
4     pakal "Eres menor de edad";  
5 }
```

Listing 4: Estructura condicional if-else

4.2 Bucle while

El bucle `while` se implementa con `muluc`:

```
1 tzolk hunab contador = 1;  
2 muluc (contador <= 5) {  
3     pakal contador;  
4     contador = contador + 1;  
5 }
```

Listing 5: Bucle while

4.3 Bucle for

El bucle `for` se implementa con `uinal`:

```
1 uinal (tzolk hunab i = 0; i < 10; i = i + 1) {  
2     pakal i;  
3 }
```

Listing 6: Bucle for

5 Funciones

5.1 Declaración de Función

Las funciones se declaran con la palabra clave `kaban`:

```
1 kaban hunab suma(hunab a, hunab b) {  
2     nik a + b;  
3 }
```

Listing 7: Declaración de función

5.2 Retorno de Valores

Para retornar un valor se utiliza la palabra clave `nik`:

```
1 kaban nawal saludo(nawal nombre) {  
2     nik "Hola, " + nombre;  
3 }
```

Listing 8: Retorno de valores

5.3 Llamada a Función

Las funciones se invocan por su nombre seguido de paréntesis:

```
1 tzolk hunab resultado = suma(5, 3);  
2 pakal resultado;
```

Listing 9: Llamada a función

6 Entrada y Salida

6.1 Salida de Datos

Para mostrar valores por pantalla se utiliza `pakal`:

```
1 pakal "Hola Mundo";  
2 pakal 42;  
3 pakal nombre;
```

Listing 10: Salida de datos

6.2 Entrada de Datos

Para leer entrada del usuario se utiliza `waaj`:

```
1 pakal " Cmo te llamas?";  
2 tzolk nawal nombre = waaj;  
3 pakal "Hola, " + nombre;
```

Listing 11: Entrada de datos

7 Análisis Léxico

El análisis léxico es la primera fase del compilador de MayaScript, encargada de transformar el texto fuente en una secuencia de tokens.

7.1 Expresiones Regulares

El analizador léxico utiliza las siguientes expresiones regulares para identificar los tokens:

7.2 Tabla de Símbolos

La tabla de símbolos es una estructura de datos que almacena información sobre los identificadores utilizados en el programa:

Token	Expresión Regular
Identificador	[a-zA-Z_] [a-zA-Z0-9_]*
Entero	[0-9]+
Decimal	[0-9]+\.[0-9]+
Cadena	"[]^" *"
Comentario de línea	//.*
Comentario multilinea	/*[\s\S]*?*/
Operadores	\+ \- * / \% \= \! \= \. . .
Delimitadores	\(\) \{ \} \[\] \; \, \.

Cuadro 5: Expresiones regulares utilizadas en el análisis léxico

Campo	Descripción
Nombre	Identificador del símbolo
Tipo	Tipo de dato (hunab, kin, nawal, bool)
Categoría	Variable, Constante o Función
Valor	Valor actual (si está asignado)
Mutable	Indica si el símbolo puede modificarse
Ámbito	Ámbito donde se define el símbolo
Línea	Línea donde se define el símbolo
Columna	Columna donde se define el símbolo

Cuadro 6: Estructura de la tabla de símbolos

7.3 Ejemplo de Tabla de Símbolos

Para el siguiente código:

```

1 mayab
2     tzolk hunab contador = 1;
3     tzolk nawal mensaje = "HOLA";
4     tun kin PI = 3.14159;
5
6     kaban hunab suma(hunab a, hunab b) {
7         nik a + b;
8     }
9 ahau

```

Listing 12: Ejemplo de código para generar tabla de símbolos

La tabla de símbolos generada sería:

Nombre	Tipo	Categoría	Valor	Mutable	Ámbito	Línea	Columna
contador	hunab	Variable	1	Sí	Global	2	17
mensaje	nawal	Variable	"Hola"	Sí	Global	3	17
PI	kin	Constante	3.14159	No	Global	4	13
suma	hunab	Función	-	No	Global	6	17
a	hunab	Variable	-	Sí	suma	6	23
b	hunab	Variable	-	Sí	suma	6	32

Cuadro 7: Ejemplo de tabla de símbolos

7.4 Tabla de Errores

La tabla de errores almacena información sobre los errores detectados durante las diferentes fases del compilador:

Campo	Descripción
Tipo	Tipo de error (Léxico, Sintáctico, Semántico)
Línea	Línea donde ocurrió el error
Columna	Columna donde ocurrió el error
Mensaje	Descripción del error
Fase	Fase del compilador donde se detectó el error

Cuadro 8: Estructura de la tabla de errores

7.5 Tipos de Errores Léxicos

Los errores léxicos que pueden ser detectados incluyen:

- Carácter no reconocido
- Identificador inválido
- Cadena sin terminar
- Comentario multilinea sin cerrar
- Número mal formado

8 Análisis Sintáctico

El análisis sintáctico es la segunda fase del compilador, encargada de verificar que la secuencia de tokens cumpla con las reglas gramaticales del lenguaje.

8.1 Gramática BNF

La gramática formal de MayaScript se define utilizando la notación BNF (Backus-Naur Form):

8.1.1. Programa

```
1 <programa> ::= "mayab" <declaraciones> "ahau"  
2 <declaraciones> ::= <declaracion> <declaraciones> |
```

Listing 13: BNF para la estructura del programa

8.1.2. Declaraciones

```
1 <declaracion> ::= <declaracion_variable> | <  
    declaracion_constante> | <declaracion_funcion> | <  
    sentencia>  
2 <declaracion_variable> ::= "tzolk" <tipo> <identificador>  
    > ["=" <expresion>] ";"  
3 <declaracion_constante> ::= "tun" <tipo> <identificador>  
    "=" <expresion> ";"  
4 <declaracion_funcion> ::= "kaban" <tipo> <identificador>  
    "(" <parametros> ")" <bloque>  
5 <parametros> ::= <lista_parametros> |  
6 <lista_parametros> ::= <parametro> | <parametro> "," <  
    lista_parametros>  
7 <parametro> ::= <tipo> <identificador>
```

Listing 14: BNF para declaraciones

8.1.3. Tipos

```
1 <tipo> ::= "hunab" | "kin" | "nawal" | "bool"
```

Listing 15: BNF para tipos de datos

8.1.4. Sentencias

```
1 <sentencia> ::= <sentencia_expresion> | <sentencia_print
  > | <bloque> | <sentencia_if> | <sentencia_while> | <
  sentencia_for> | <sentencia_return>
2 <sentencia_expresion> ::= <expresion> ";"
3 <sentencia_print> ::= "pakal" <expresion> ";"
4 <bloque> ::= "{" <declaraciones> "}"
5 <sentencia_if> ::= "chac" "(" <expresion> ")" <sentencia
  > ["imix" <sentencia>]
6 <sentencia_while> ::= "muluc" "(" <expresion> ")" <
  sentencia>
7 <sentencia_for> ::= "uinal" "(" (<declaracion_variable>
  | <sentencia_expresion> | ";") [<expresion>] ";" [<
  expresion>] ")" <sentencia>
8 <sentencia_return> ::= "nik" [<expresion>] ";"
```

Listing 16: BNF para sentencias

8.1.5. Expresiones

```
1 <expresion> ::= <asignacion>
2 <asignacion> ::= <identificador> "=" <asignacion> | <
  logica_or>
3 <logica_or> ::= <logica_and> | <logica_and> "||" <
  logica_or>
4 <logica_and> ::= <igualdad> | <igualdad> "&&" <
  logica_and>
5 <igualdad> ::= <comparacion> | <comparacion> ("!=" |
  "==") <igualdad>
6 <comparacion> ::= <termino> | <termino> ">" | ">=" |
  "<" | "<=") <comparacion>
7 <termino> ::= <factor> | <factor> ("+" | "-") <termino>
8 <factor> ::= <unario> | <unario> ("*" | "/" | "%") <
  factor>
9 <unario> ::= ("!" | "-") <unario> | <llamada>
```

```

10 <llamada> ::= <primaria> | <primaria> "(" <argumentos>
    ")"
11 <argumentos> ::= <lista_argumentos> |
12 <lista_argumentos> ::= <expresion> | <expresion> "," <
    lista_argumentos>
13 <primaria> ::= <literal> | <identificador> | "(" <
    expresion> ")" | "waaj"

```

Listing 17: BNF para expresiones

8.1.6. Literales y Tokens básicos

```

1 <literal> ::= <numero> | <cadena> | "haab" | "manik"
2 <numero> ::= <entero> | <decimal>
3 <entero> ::= <digito>+
4 <decimal> ::= <digito>+ "." <digito>+
5 <cadena> ::= "\"" <caracteres> "\""
6 <caracteres> ::= <caracter>*
7 <identificador> ::= <letra> (<letra> | <digito> | "_")*
8 <letra> ::= "a" | "b" | ... | "z" | "A" | "B" | ... | "Z"
    "
9 <digito> ::= "0" | "1" | ... | "9"

```

Listing 18: BNF para literales y tokens básicos

8.2 Árboles de Derivación

Un árbol de derivación es una representación gráfica de cómo la gramática genera una cadena del lenguaje. A continuación, se presentan ejemplos de árboles de derivación para diferentes construcciones en MayaScript.

8.2.1. Ejemplo 1: Declaración de variable con asignación

Código: `tzolk hunab x = 42;`

8.2.2. Ejemplo 2: Sentencia if-else

Código:

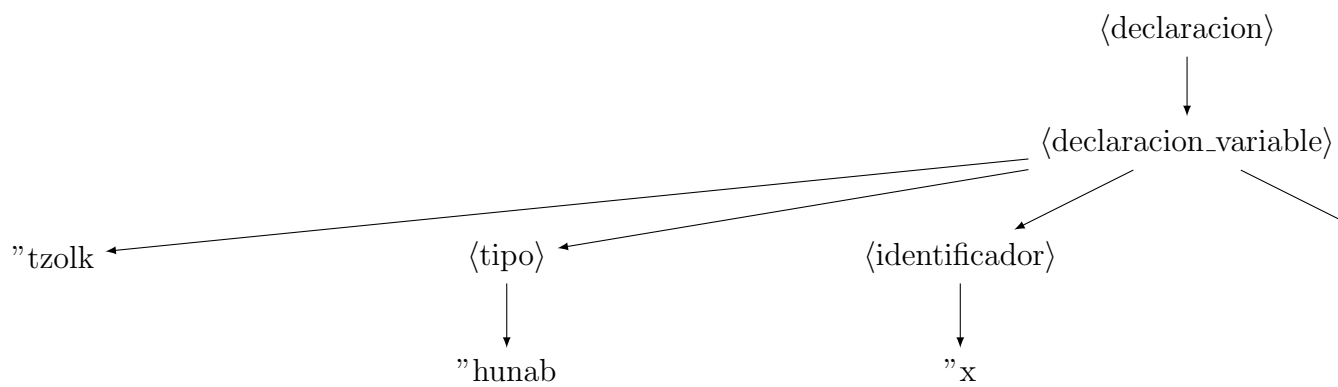


Figura 1: Árbol de derivación para la declaración de variable

```

1  chac (x > 10) {
2      pakal "Mayor";
3  } imix {
4      pakal "Menor o igual";
5  }

```

8.2.3. Ejemplo 3: Bucle while

Código:

```

1  muluc (i < 5) {
2      pakal i;
3      i = i + 1;
4  }

```

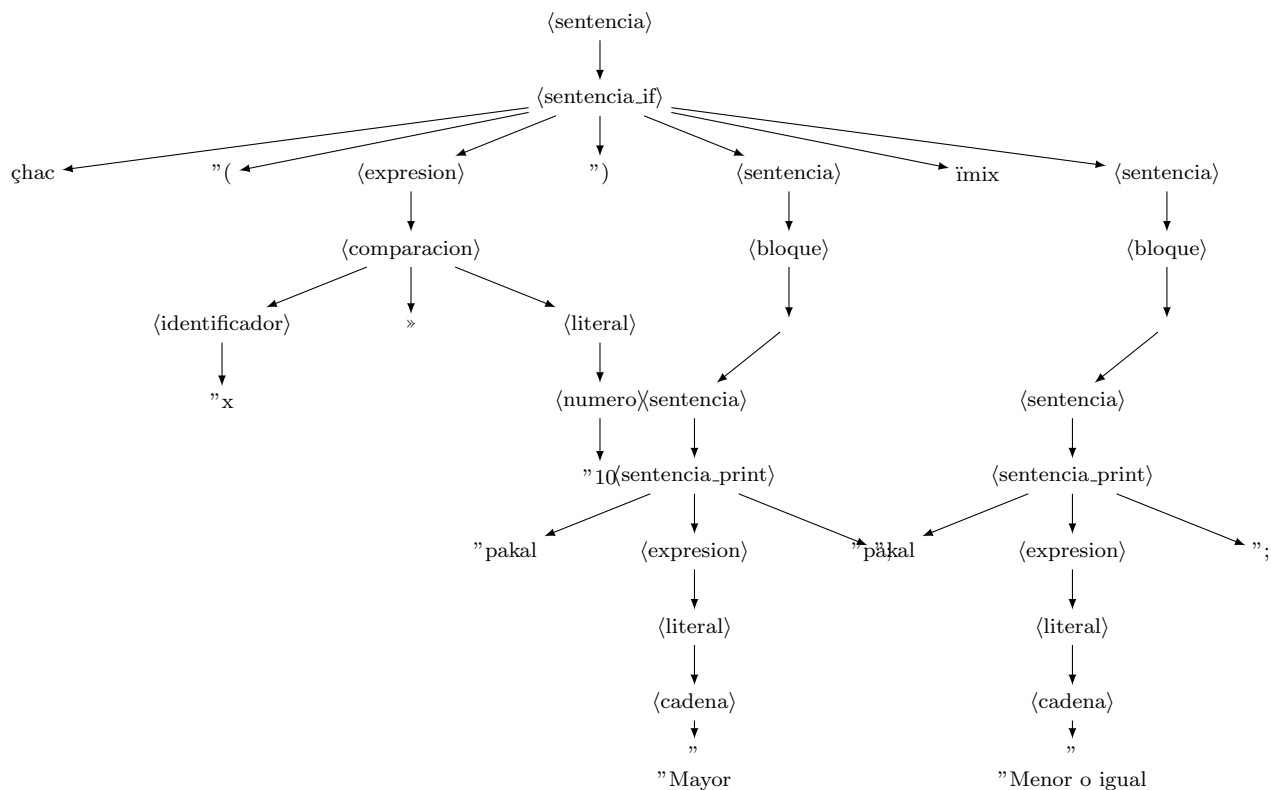


Figura 2: Árbol de derivación para la sentencia if-else

8.3 Tipos de Errores Sintácticos

El análisis sintáctico puede detectar los siguientes tipos de errores:

- Falta de tokens esperados (ej. punto y coma, paréntesis, llaves)
- Tokens inesperados
- Estructuras incorrectas (ej. declaración de variable mal formada)
- Fin de archivo inesperado

8.4 Ejemplo de Tabla de Errores

Para el siguiente código con errores:

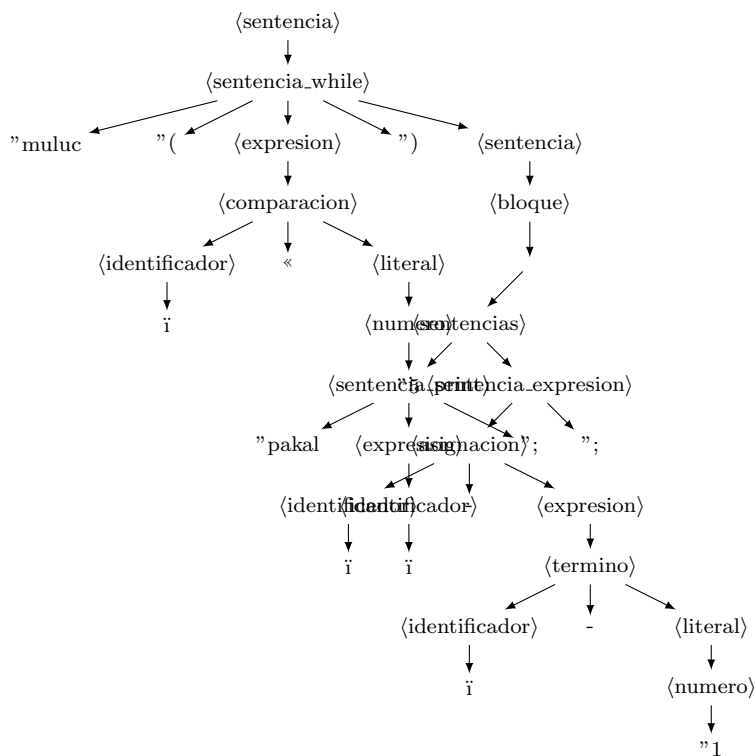


Figura 3: Árbol de derivación para el bucle while

```

1 mayab
2     tzolk hunab x = 10
3     tzolk kin y = "texto";
4     z = 30;
5 ahau

```

Listing 19: Código con errores

La tabla de errores generada sería:

Tipo	Línea	Columna	Mensaje	Fase
Sintáctico	2	19	Se esperaba ';' después de la declaración de variable	Análisis
Semántico	3	15	Tipo incompatible: no se puede asignar nawal a kin	Análisis
Semántico	4	5	Variable 'z' no declarada	Análisis

Cuadro 9: Ejemplo de tabla de errores

9 Palabras Reservadas

MayaScript cuenta con las siguientes palabras reservadas, inspiradas en términos de la cultura maya:

Palabra	Significado	Uso
mayab	Inicio del programa	Delimita el inicio del código fuente
ahau	Fin del programa	Delimita el fin del código fuente
tzolk	Declaración de variable	Para crear variables
tun	Declaración de constante	Para crear constantes
hunab	Tipo entero	Para variables o constantes enteras
kin	Tipo flotante	Para variables o constantes decimales
nawal	Tipo cadena	Para variables o constantes de texto
bool	Tipo booleano	Para variables o constantes lógicas
haab	Verdadero	Valor booleano true
manik	Falso	Valor booleano false
chac	Condición if	Para estructuras de decisión
imix	Condición else	Para la alternativa en if
muluc	Bucle while	Para ciclos condicionales
uinal	Bucle for	Para ciclos controlados por contador
kaban	Función	Para declarar funciones
nik	Retorno	Para devolver valores de funciones
pakal	Imprimir	Para mostrar valores en pantalla
waaj	Entrada	Para leer valores del usuario

Cuadro 10: Palabras reservadas de MayaScript

10 Ejemplos Completos

A continuación, se presentan ejemplos completos que muestran las diferentes características de MayaScript en acción.

10.1 Ejemplo 1: Calculadora Simple

```
1 mayab
2     pakal "Calculadora Simple";
```

```

3
4 // Entrada de datos
5 pakal "Ingrese primer n mero:";
6 tzolk hunab num1 = waaj;
7
8 pakal "Ingrese segundo n mero:";
9 tzolk hunab num2 = waaj;
10
11 // Operaciones
12 tzolk hunab suma = num1 + num2;
13 tzolk hunab resta = num1 - num2;
14 tzolk hunab mult = num1 * num2;
15 tzolk hunab div = num1 / num2;
16
17 // Mostrar resultados
18 pakal "Suma: " + suma;
19 pakal "Resta: " + resta;
20 pakal "Multiplicaci n: " + mult;
21 pakal "Divisi n: " + div;
22 ahau

```

Listing 20: Calculadora Simple

10.2 Ejemplo 2: Factorial

```

1 mayab
2 // Funci n factorial
3 kaban hunab factorial(hunab n) {
4     chac (n <= 1) {
5         nik 1;
6     } imix {
7         nik n * factorial(n - 1);
8     }
9 }
10
11 // Probar la funci n
12 pakal "Ingrese un n mero para calcular su factorial
13 :";
14 tzolk hunab num = waaj;

```

```

15     chac (num < 0) {
16         pakal "El n mero debe ser positivo";
17     } imix {
18         tzolk hunab resultado = factorial(num);
19         pakal "El factorial de " + num + " es " +
resultado;
20     }
21 ahau

```

Listing 21: Cálculo de factorial recursivo

11 Pruebas

11.1 Pruebas con Código Correcto

Se han realizado diversas pruebas con código válido para verificar el correcto funcionamiento del compilador. Estas pruebas incluyen:

- Declaraciones de variables y constantes
- Uso de operadores aritméticos y lógicos
- Estructuras de control (if-else, while, for)
- Definición y llamada a funciones
- Entrada y salida de datos

11.2 Pruebas con Código Erróneo

También se han realizado pruebas con código que contiene errores para verificar que el compilador los detecta correctamente:

- Errores léxicos: caracteres no reconocidos, cadenas sin terminar, etc.
- Errores sintácticos: estructura incorrecta, falta de tokens, etc.
- Errores semánticos: variables no declaradas, tipos incompatibles, etc.

12 Modo Interactivo

MayaScript incluye un modo interactivo que permite escribir y ejecutar código línea por línea:

```
1 ./mayascript.sh
```

Listing 22: Inicio del modo interactivo

O directamente:

```
1 java -jar target/mayascript-1.0-SNAPSHOT.jar
```

12.1 Ejemplo de uso interactivo

```
1 > mayab
2 >     pakal "Hola desde el modo interactivo";
3 > ahau
```

Listing 23: Ejemplo de uso del modo interactivo

13 Limitaciones Actuales

Esta implementación de MayaScript tiene las siguientes limitaciones:

- No se soporta la programación orientada a objetos
- No hay soporte para manejo de excepciones
- No se incluye una biblioteca estándar extensa
- El sistema de tipos es estático y simple

14 Conclusiones

MayaScript es un proyecto educativo diseñado para demostrar los principios fundamentales de los compiladores. A través de su sintaxis única inspirada en la cultura maya, proporciona una plataforma para aprender sobre:

- Análisis léxico y reconocimiento de tokens

- Análisis sintáctico y gramáticas formales
- Tablas de símbolos y manejo de errores
- Ejecución e interpretación de código

Aunque tiene limitaciones en comparación con lenguajes de programación de producción, su enfoque educativo lo convierte en una herramienta valiosa para comprender los conceptos fundamentales de la compilación e interpretación de lenguajes de programación.